A view of 0 or 1 elements: `view::maybe`

- Document number: P1255R1
- Date: 2018-11-26

- Author: Steve Downey <sdowney2@bloomberg.net>, <sdowney@gmail.com>

- Audience: LEWG

Abstract: This paper proposes `view::maybe` a range adaptor that produces a view with cardinality 0 or 1 which adapts nullable types such as `std::optional` and pointer types.

Table of Contents

1 Changes since R0

1.1 Remove customization point objects

Removed view::maybe_has_value and view::maybe_value, instead requiring that the nullable type be dereferenceable and contextually convertable to bool.

1.2 Concept `Nullable`, for exposition

Concept Nullable, which is Readable and contextually convertable to bool

1.3 Capture rvalues by decay copy

Hold a copy when constructing a view over a nullable rvalue.

1.4 Remove maybe_view as a specified type

Introduced two exposition types, one safely holding a copy, the other referring to the nullable

2 Motivation

In writing range transformation pipelines it is useful to be able to lift a nullable value into a view that is either empty or contains the value held by the nullable. The adapter `view::single` fills a similar purpose for non-nullable values, lifting a single value into a view, and `view::empty` provides a range of no values of a given type. A `view::maybe` adaptor also allows nullable values to be treated as ranges when it is otherwise undesirable to make them containers, for example `std::optional`.

```
std::vector<std::optional<int>> v{
  std::optional<int>{42},
  std::optional<int>{},
  std::optional<int>{6 * 9}};

auto r = view::join(view::transform(v, view::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 42
}
```

In addition to range transformation pipelines, `view::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an if statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the `if` is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (*opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK
    }
}

for (auto&& opt : view::maybe(possible_value())) {
    // a few dozen lines ...
    use(opt); // opt is OK
}
```

3 Proposal

Add a range adaptor object `view::maybe`, returning a view over a nullable object, capturing by value temporary nullables. A `Nullable` object is one that is both contextually convertable to bool and which models the `Readable` concept. Non void pointers, `std::optional`, and the proposed `outcome` and `expected` types all model Nullable.

4 Design

The basis of the design is to hybridize `view::single` and `view::empty`. If the underlying object claims to hold a value, as determined by checking if the object when converted to bool is true, `begin` and `end` of the view are equivalent to the address of the held value within the underlying object and one past the underlying object. If the underlying object does not have a value, `begin` and `end` return `nullptr`.

The `view::maybe` range adapter object will create either a safe view, containing a move initialized decay_copy of the nullable, or a reference view, referring to the nullable value, depending on the deduced referenceness of the template parameter.

5 Synopsis

```
namespace std::ranges {

// For Exposition
template<class> struct dereference_type {};
```

```
template<class D>
requires
requires(const D& d) {{ *d } -> auto&&; }
struct dereference_type<D> {
    using type = decltype(*declval<const D&>());
};

template<class D>
using dereference_t = typename dereference_type<D>::type;


template<class T>
concept bool ContextualBool =
    requires(const T& t) {
    {bool(t)} -> bool;
};

template <class T>
concept bool Nullable =
    Readable<remove_reference_t<T>> &&
    ContextualBool<T> &&
    requires (const T& t) {
    typename dereference_t<T>;
    is_object_v<dereference_t<T>>;
};


template <Nullable Maybe>
requires is_object_v<R>
class safe_maybe_view
    : public view_interface<safe_maybe_view<Maybe>> {
  private:
    using T = decay_t<remove_reference_t<dereference_t<Maybe>>>;
    using M = remove_cv_t<remove_reference_t<Maybe>>;

    semiregular_box<M> value_;

  public:
    safe_maybe_view() = default;

    constexpr explicit safe_maybe_view(Maybe const& maybe);
    constexpr explicit safe_maybe_view(Maybe&& maybe);
    constexpr T*       begin() noexcept;
    constexpr const T* begin() const noexcept;
    constexpr T*       end() noexcept ;

    constexpr const T* end() const noexcept ;

    constexpr ptrdiff_t size() noexcept ;

    constexpr T* data() noexcept ;

    constexpr const T* data() const noexcept;
};

template <Nullable Maybe>
requires is_object_v<R>
class ref_maybe_view
    : public view_interface<ref_maybe_view<Maybe>> {
    remove_reference_t<Maybe>* value_;
    using R = remove_reference_t<decltype(**value_)>;

  public:
    constexpr ref_maybe_view() = default;
```

```
    constexpr ref_maybe_view(ref_maybe_view const&) = default;

    constexpr explicit ref_maybe_view(Maybe& maybe);

    constexpr R*       begin() noexcept;
    constexpr const R* begin() const noexcept;
    constexpr R*       end() noexcept ;

    constexpr const R* end() const noexcept;

    constexpr ptrdiff_t size() noexcept;

    constexpr R* data() noexcept;

    constexpr const R* data() const noexcept;
};

namespace view {
struct __maybe_fn {
    template <Nullable T>
    constexpr auto operator()(T&& t) const
        noexcept(noexcept(ref_maybe_view{std::forward<T>(t)}))
        requires std::is_reference_v<T> &&
        requires {ref_maybe_view{std::forward<T>(t)};}
    {
        return ref_maybe_view{std::forward<T>(t)};
    }

    template <Nullable T>
    constexpr auto operator()(T&& t) const
        noexcept(noexcept(safe_maybe_view{std::forward<T>(t)}))
        requires !std::is_reference_v<T> &&
        requires {safe_maybe_view{std::forward<T>(t)};}
    {
        return safe_maybe_view{std::forward<T>(t)};
    }
};

inline constexpr __maybe_fn maybe{};

} // namespace view
} // namespace std::ranges
```

[Example:

```
  optional o{4};
  for (int i : view::maybe(o))
    cout << i; // prints 4

  maybe_view e{ };
  for (int i : view::maybe(std::optional{}))
    cout << i; // does not print

  int        j  = 8;
  int*       pj = &j;
  for (auto i : view::maybe(pj))
    std::cout << i; // prints 8
```

— end example]

5.1 `view::maybe`

`view::maybe` returns a View over a Nullable that is either empty if the nullable is empty, or includes the contents of the nullable obhect.

The name view::maybe denotes a customization point object ([customization.point.object]). The expression view::maybe(E) for some subexpression E is expression-equivalent to :

– safe_maybe_view{E}, if the expression is well formed, where E is decay copied into the safe_maybe_view

– otherwise ref_maybe_view{}, if that expression is well formed, where ref_maybe_view refers to E

– otherwise view::maybe(E) is ill-formed

Note: Whenever view::maybe(E) is a valid expression, it is a prvalue whose type models View. — end note]

6 Impact on the standard

A pure library extension, affecting no other parts of the library or language.

7 References

[P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella. The One Ranges Proposal URL: https://wg21.link/p0896r3

[P0323R7] Vicente Botet, JF Bastien. std::expected URL: https://wg21.link/p0323r7